

BugVault Technical Specification

End-to-end production documentation for **BugVault** — a full-stack personal bug tracking, root-cause logging, and solution reference web application built with Node.js, Express, SQLite, and modern web standards.

Author: Venkata Ajay Kumar
GitHub: @venkataajaykumar19
Repository: venkataajaykumar19/BugVault
Date: September 16, 2026
Local Tests: 17/17 Assertions Passing

SECTION 1 System Architecture

BugVault is architected as a lightweight, decoupled full-stack client-server system designed to solve a developer problem: forgetting root causes and solutions for repetitive programming bugs. The application adheres to strict minimalism, zero bloated frameworks, and robust data persistence.

1. WHERE IS THE FRONTEND DEPLOYED?

GitHub Pages (Static Web Hosting)

Served via HTTPS from `https://venkataajaykumar19.github.io/BugVault/frontend/index.html`

2. WHERE IS THE BACKEND DEPLOYED?

Render (Cloud Web Service)

Hosted at `https://bugvault-api.onrender.com` running Node.js runtime environment.

3. WHAT DATABASE IS USED?

SQLite 3 (Embedded Relational Engine)

Directly driven via native `better-sqlite3` (v11.8.1) with Write-Ahead Logging (WAL).

4. WHERE IS THE DATABASE HOSTED?

Co-located on Server Filesystem

Stored locally in `backend/database.sqlite` on the backend host with atomic durability.

5 & 6. DOES BACKEND CALL EXTERNAL APIS? WHICH EXTERNAL APIS?

No External APIs Called (100% Self-Contained)

The backend performs zero outbound HTTP requests. It relies entirely on internal business logic, schema validation, and local SQLite persistence, ensuring zero third-party latency, zero rate-limiting vulnerabilities, and sub-millisecond query execution.

— SYSTEM COMPONENT & DATA FLOW DIAGRAM —

User interacts via Web Browser (Desktop / Mobile)

FRONTEND CLIENT (UI)

HTML5 Semantic Structure • Responsive CSS3 • Vanilla ES6+ JavaScript

DEPLOYED ON: GITHUB PAGES

URL: `https://venkataajaykumar19.github.io/BugVault/frontend/index.html`

HTTPS / REST JSON API Requests (CORS Enabled)

EXPRESS REST API SERVER (BACKEND)

Node.js (v18+) • Express 4.21.2 • cors • dotenv • Prepared Statements

DEPLOYED ON: RENDER WEB SERVICE

URL: `https://bugvault-api.onrender.com` (Port 3000)

Synchronous In-Process C++ Bindings (better-sqlite3)

SQLITE 3 PERSISTENT DATABASE

WAL Mode Enabled • Table: bugs • Co-located Single-File Storage

STORAGE: BACKEND/DATABASE.SQLITE

Step-by-Step Architecture Flow (End-to-End User Action Lifecycle)

The complete path traversed by data from user interaction to disk storage and back to visual feedback follows a deterministic, 10-step lifecycle:

- 1 **User Interaction:** The user opens the web client and fills in the bug entry form in `frontend/index.html` (inputs: `#bug-title`, `#bug-error`, `#bug-cause`, `#bug-solution`, `#bug-date`), then clicks `#submit-btn`.
- 2 **Client Event Capture:** In `frontend/script.js`, the submit listener on `#bug-form` intercepts the form submission via `e.preventDefault()` to prevent native page refresh.
- 3 **Client Validation & Serialization:** `frontend/script.js` extracts trimmed values, verifies that no field is blank, activates the loading spinner, disables the submit button, and serializes the payload into JSON: `{ title, error, cause, solution, date }`.
- 4 **HTTP Request Dispatch:** The client issues an asynchronous `fetch()` request targeting `${BACKEND_URL}/bugs` with HTTP method `POST`, header `Content-Type: application/json`, and the stringified JSON payload.
- 5 **Network Ingestion & Middleware:** The Render proxy routes the HTTPS request to the Node.js Express process. Express in `backend/server.js` invokes `cors()` (validating cross-origin headers) followed by `express.json()` (parsing JSON into `req.body`).
- 6 **Controller Validation:** The route handler `app.post('/bugs', ...)` enforces that all five required fields exist and are non-empty strings. If invalid, it returns HTTP 400 immediately.
- 7 **Database Execution:** The handler executes the pre-compiled SQLite statement `insertBugStmt.run(...)` via `better-sqlite3`, inserting the record into table `bugs` in `backend/database.sqlite` and writing to the WAL journal.
- 8 **Record Retrieval & Response:** The handler fetches the newly created record via `selectBugByIdStmt.get(info.lastInsertRowid)` and returns it as a JSON payload with HTTP status `201 Created`.
- 9 **Client Ingestion & Notification:** The client promise resolves. `frontend/script.js` displays a green alert banner (`#alert-banner`) reading "Bug record #{id} saved successfully!", resets form fields, and sets today's date.
- 10 **DOM Reconciliation:** The frontend invokes `fetchBugs()`, which calls `GET /bugs`. The returned JSON array is rendered by `renderBugsList()` into styled bug cards inside `#bugs-container`, updating the badge count.

SECTION 2 API Documentation

The BugVault backend exposes a strictly scoped REST API implemented in `backend/server.js`. The backend registers exactly **5 HTTP routes**: 1 operational health check endpoint and 4 core CRUD application endpoints.

Route Classification & Separation of Concerns:

GET /health serves exclusively as an infrastructure liveness/health probe and does NOT touch the database entities. The remaining 4 routes (POST /bugs , GET /bugs , PUT /bugs/:id , DELETE /bugs/:id) represent the complete application resource contract.

OPERATIONAL / HEALTH

GET /health

Server uptime & liveness check

Purpose:

Verifies that the Express runtime process is active and responsive. Queried on frontend load by `checkBackendHealth()` in `frontend/script.js` to update the UI status badge (`#backend-status-badge`).

Request Headers:

Accept: application/json • Request Body: None

Status Code	Meaning	Response Payload
200 OK	Service is healthy and ready to accept traffic.	<code>{ "status": "ok" }</code>
500 Server Error	Service encountered an unhandled process failure.	<code>{ "error": "Internal Server Error", "message": "... " }</code>

Purpose: Validates payload fields and inserts a new bug record into the SQLite database table `bugs`.

Request Headers: `Content-Type: application/json`, `Accept: application/json`

Field	Type	Required	Description
<code>title</code>	string	Yes	Concise summary of the bug/error (non-empty)
<code>error</code>	string	Yes	Raw error message, stack trace snippet, or symptom
<code>cause</code>	string	Yes	Root cause explaining why the error occurred
<code>solution</code>	string	Yes	Step-by-step resolution or code modifications applied
<code>date</code>	string	Yes	Date when encountered in ISO format (<code>YYYY-MM-DD</code>)

```
{
  "title": "Prisma environment error",
  "error": "Environment variable not found: DATABASE_URL",
  "cause": "DATABASE_URL was missing from .env after fresh git clone",
  "solution": "Added DATABASE_URL to backend/.env and ran npx prisma generate",
  "date": "2026-09-16"
}
```

Status	Condition	Response Format
201 Created	Record created successfully	Returns created bug object including autoincremented integer <code>id</code> .
400 Bad Request	Missing or empty required fields	<code>{ "error": "Missing required fields", "missingFields": ["title"], "message": " ... " }</code>
500 Server Error	Database insertion failure	<code>{ "error": "Failed to create bug record", "message": "A database error occurred." }</code>

Purpose: Queries SQLite for all stored bug records, ordered by `id DESC` (newest first).

Request Headers: `Accept: application/json` • **Request Body:** None

Status	Condition	Response Format
200 OK	Records retrieved	JSON array: <code>[{ "id": 1, "title": " ... ", "error": " ... ", "cause": " ... ", "solution": " ... ", "date": " ... " }]</code>
500 Server Error	Database query failure	<code>{ "error": "Failed to fetch bug records", "message": "A database error occurred." }</code>

Purpose: Updates all fields of an existing bug identified by URL parameter `:id`.

URL Parameters: `id` (Integer, required): The unique record ID.

Headers: `Content-Type: application/json`, `Accept: application/json`

```
{
  "title": "Prisma environment error (Resolved)",
  "error": "Environment variable not found: DATABASE_URL",
  "cause": "DATABASE_URL was missing from .env after fresh git clone",
  "solution": "Added DATABASE_URL to backend/.env and ran npx prisma generate",
  "date": "2026-09-16"
}
```

Status	Condition	Response Format
200 OK	Update successful	Returns complete updated bug object.
400 Bad Request	Non-integer ID or missing required fields	<code>{ "error": "Invalid bug ID", "message": "Bug ID must be a positive integer." }</code>
404 Not Found	Bug ID does not exist	<code>{ "error": "Bug record not found", "message": "No bug found with ID 99." }</code>
500 Server Error	Database update failure	<code>{ "error": "Failed to update bug record", "message": "A database error occurred." }</code>

Purpose: Removes the specified bug record permanently from the SQLite database table `bugs`.

URL Parameters: `id` (Integer, required): The unique record ID • **Headers:** `Accept: application/json`

Status	Condition	Response Format
200 OK	Deletion successful	<code>{ "success": true, "message": "Bug with ID 1 successfully deleted.", "deletedId": 1 }</code>
400 Bad Request	Non-integer ID parameter	<code>{ "error": "Invalid bug ID", "message": "Bug ID must be a positive integer." }</code>
404 Not Found	No record found with specified ID	<code>{ "error": "Bug record not found", "message": "No bug found with ID 99." }</code>
500 Server Error	Database deletion failure	<code>{ "error": "Failed to delete bug record", "message": "A database error occurred." }</code>

Global Catch-All Error Handling Middleware:

In `backend/server.js`, an Express error-handling middleware intercepts malformed JSON syntax errors in request bodies and returns an immediate HTTP 400 Bad Request: `{ "error": "Malformed JSON", "message": "The request payload contains invalid JSON syntax." }`.

SECTION 3 Database Design

BugVault uses **SQLite 3** driven by `better-sqlite3` (v11.8.1). SQLite was chosen intentionally over client-server engines because it provides zero configuration overhead, instantaneous sub-millisecond execution, complete file portability, and ACID transaction guarantees.

Assignment Requirement Audit & Blocker Notification:

The Challenge #10 assignment specification requires the Database Design section to document *at least two entities/tables with fields, types, and relationships*.

An empirical audit of the actual BugVault codebase (`backend/server.js` and `backend/database.sqlite`) reveals that the project was intentionally built as an MVP containing **exactly ONE database entity: bugs** . In strict adherence to the assignment's non-fabrication rule ("DO NOT invent a fake database entity just to satisfy the requirement"), this report accurately documents the single existing entity and flags the absence of a second entity as a formal assignment blocker.

Property	Setting	Technical Justification
Database Engine	SQLite 3	Embedded, serverless SQL database stored co-located with the Node.js runtime process.
Driver / Library	<code>better-sqlite3</code> (v11.8.1)	Synchronous, high-throughput execution with direct V8 memory bindings. Outperforms async drivers for local I/O.
Journal Mode	<code>PRAGMA journal_mode = WAL;</code>	Write-Ahead Logging permits readers to read while writers write, significantly enhancing concurrency.
File Storage	<code>backend/database.sqlite</code>	Single-file database with accompanying <code>-wal</code> (journal) and <code>-shm</code> (shared memory index) files.
Migration Strategy	Automated DDL at Startup	Executed synchronously on server initialization via <code>db.exec(CREATE TABLE IF NOT EXISTS ...)</code> .

Entity 1 (Actual / Implemented): Table `bugs`

The single actual table implemented in the database:

Column Name	Data Type	Constraints	Default	Description
<code>id</code>	INTEGER	PRIMARY KEY AUTOINCREMENT	Auto-incremented	Unique surrogate key identifying each bug entry.
<code>title</code>	TEXT	NOT NULL	None	Concise headline summarizing the problem.
<code>error</code>	TEXT	NOT NULL	None	Full error message, stack trace snippet, or symptom.
<code>cause</code>	TEXT	NOT NULL	None	Technical explanation of the underlying root cause.
<code>solution</code>	TEXT	NOT NULL	None	Detailed fix, resolution steps, or code patch applied.
<code>date</code>	TEXT	NOT NULL	None	ISO 8601 calendar date string (<code>YYYY-MM-DD</code>).

Compiled Prepared Statements & Injection Prevention

All queries use pre-compiled prepared statements compiled once during server bootstrap in `backend/server.js` , eliminating SQL injection risks through parameter binding:

```
-- 1. Insert Statement (Named Parameter Binding)
INSERT INTO bugs (title, error, cause, solution, date)
VALUES (@title, @error, @cause, @solution, @date);

-- 2. Select All Statement (Ordered Chronologically Descending)
SELECT id, title, error, cause, solution, date FROM bugs ORDER BY id DESC;

-- 3. Select by ID Statement (Positional Parameter Binding)
SELECT id, title, error, cause, solution, date FROM bugs WHERE id = ?;

-- 4. Update Statement (Named Parameter Binding)
UPDATE bugs
SET title = @title, error = @error, cause = @cause, solution = @solution, date = @date
WHERE id = @id;

-- 5. Delete Statement (Positional Parameter Binding)
DELETE FROM bugs WHERE id = ?;
```

Architectural Design for Entity 2 (Proposed Extension: categories)

If BugVault is extended to fulfill the multi-entity assignment requirement without altering the current codebase, the natural second entity is a categories (or tags) table to categorize errors (e.g., Syntax, Environment, Database, Network):

Column Name	Data Type	Constraints	Relationship	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	None	Unique category identifier.
name	TEXT	NOT NULL UNIQUE	None	Category label (e.g., "Prisma", "CORS", "Docker").
description	TEXT	NULL	None	Optional description of the error category.

Relationship Specification: In this design, bugs would acquire a foreign key column category_id INTEGER REFERENCES categories(id), establishing a One-to-Many relationship (One category has many bugs; each bug belongs to one category). Note: This relationship is documented strictly as a design blueprint and is NOT present in the current BugVault MVP.

SECTION 4 Deployment Documentation

BugVault is deployed across two cloud platforms following a decoupled production architecture: GitHub Pages for static frontend assets and Render for the dynamic Node.js backend Web Service.

Deployment Metadata & URLs

Layer	Hosting Platform	Live Production URL	Configuration File
Frontend	GitHub Pages (Static)	https://venkataajaykumar19.github.io/BugVault/frontend/index.html	frontend/index.html
Backend	Render (Web Service)	https://bugvault-api.onrender.com	render.yaml
Health Check	Render Endpoint	https://bugvault-api.onrender.com/health	backend/server.js
Repository	GitHub (Public)	https://github.com/venkataajaykumar19/BugVault	Git repository root

Important Production Storage Clarification (Render Free Tier):

On the Render Free Tier Web Service, the filesystem is ephemeral

. Any writes to backend/database.sqlite will persist during an active container lifecycle, but will reset upon container restarts or re-deployments unless a paid Render Persistent Disk is attached. For permanent, guaranteed persistent storage, BugVault is fully verified on local disk storage (where database.sqlite persists indefinitely across restarts).

Infrastructure-as-Code (Render Web Service)

Declared in root render.yaml :

```
services:
  - type: web
    name: bugvault-api
    env: node
    plan: free
    rootDir: backend
    buildCommand: npm install
    startCommand: node server.js
    envVars:
      - key: PORT
        value: 3000
```

Dynamic Client Environment Switching

In frontend/script.js , client dynamically toggles between local backend and Render deployment:

```
const isLocal = window.location.hostname === 'localhost' ||
  window.location.hostname === '127.0.0.1' ||
  window.location.protocol === 'file:';

const BACKEND_URL = isLocal
  ? 'http://localhost:3000'
  : 'https://bugvault-api.onrender.com';
```

Environment Variable Configuration

Variable	Default	Location	Security Rule
PORT	3000	backend/.env • Render Dashboard	Documented in backend/.env.example (tracked). Live .env file is strictly excluded by .gitignore.

Local Reproduction & Execution Guide

```
# 1. Clone repository and install backend dependencies
git clone https://github.com/venkataajaykumar19/BugVault.git
cd BugVault/backend
npm install

# 2. Configure environment and start API server
cp .env.example .env
npm start
# Server listens at http://localhost:3000 (Test: curl http://localhost:3000/health)

# 3. Launch Frontend Client
# Option A: Open directly in Chrome/Edge:
start ../frontend/index.html
# Option B: Run static file server:
npx serve ../frontend

# 4. Execute Automated Verification Suite
node test_crud.js
# Runs all 17 automated assertions covering Health, POST, GET, PUT, DELETE, and Edge Cases
```

SECTION 5 Codebase Structure

The BugVault repository is organized into cleanly partitioned client and server directories, enforcing strict separation of concerns between presentation, business logic, persistence, and deployment infrastructure.

Annotated Directory Tree (Matches Active Git Repository)

```
BugVault/ |— .gitignore # Excludes node_modules, .env, *.sqlite*, OS cache |— BugVault_Technical_Documentation.pdf #
Challenge #10 comprehensive technical documentation |— README.md # Project overview, schema, routes, live URLs, setup
|— VIDEO_SCRIPT.md # 3-minute presentation script for video submission |— index.html # Root redirect to
frontend/index.html |— render.yaml # Render Web Service Infrastructure-as-Code descriptor |— serve_frontend.js #
Utility script to serve frontend files locally |— verify_frontend_browser.js # Puppeteer / browser automation test
script |— backend/ # Node.js + Express REST API & SQLite database | |— .env.example # Environment template tracking
PORT=3000 | |— package-lock.json # Deterministic dependency lockfile | |— package.json # Dependencies: express, cors,
dotenv, better-sqlite3 | |— server.js # Express server, SQLite setup, 5 routes, error middleware | |— test_crud.js #
Comprehensive automated test suite (17 assertions) |— frontend/ # Static web client (GitHub Pages root) | |—
index.html # Accessible HTML5 structure, semantic form, and cards grid | |— script.js # CRUD state management, DOM
reconciliation, XSS sanitization | |— style.css # Responsive styling, design tokens, flexbox/grid layout |—
[Verification Screenshots] |— frontend_verified_initial.png # Browser evidence: initial loaded state |—
frontend_verified_created.png # Browser evidence: record creation |— frontend_verified_final.png # Browser evidence:
final CRUD verified state
```

File Responsibilities & Architectural Boundaries

File Path	Layer	Key Responsibilities
backend/server.js	Backend (Core)	Initializes Express, enables CORS, opens SQLite database, runs table migrations, compiles prepared SQL statements, registers 5 routes, and handles global errors.
backend/test_crud.js	Testing / QA	Autonomous test suite using native Node.js HTTP client to verify health check, all CRUD routes, error codes (400, 404), and data persistence across sequential operations.
frontend/index.html	Frontend (Structure)	Semantic HTML5 document featuring header badge, global alert banner, two-column split layout (entry form on left, bug cards on right), and loading/empty states.
frontend/script.js	Frontend (Logic)	Manages client state (bugs array, editingBugId), handles form submission, toggles edit mode, dispatches REST API requests, and renders bug cards.
frontend/style.css	Frontend (Styling)	CSS3 design system using HSL color variables, modern typography, elevation drop shadows, animated loading spinner, and responsive media queries.
render.yaml	DevOps / Release	Declares cloud infrastructure for Render: free tier web service, root directory backend , build command, and startup command.

Engineering Security & Robustness Practices

Vector	Implementation in BugVault	Verification Result
SQL Injection	100% prepared statements with parameterized inputs via better-sqlite3 . No string concatenation.	Verified in backend/test_crud.js (SQL strings treated strictly as literal data).
XSS Prevention	All user strings passed through escapeHtml() in frontend/script.js before DOM injection.	Verified (HTML entities & , < , > , " , ' safely escaped).
Secret Protection	.env file gitignored. Only safe template .env.example tracked in git.	Verified via git status and repository file audit (zero secrets committed).
Defensive JSON Parsing	Global Express error handler catches syntax errors from malformed JSON and responds with HTTP 400.	Verified (invalid JSON payloads handled cleanly without process termination or stack trace leaks).